

CS201 Final

11 August 2025

Olcay Taner YILDIZ

Abstract— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted,

- (i) The input data structure is not empty
- (ii) Time complexity of the algorithm does not matter
- (iii) You can not use extra data structures including arrays
- (iv) You can use any function given in the data structure
- (v) You can not modify the data structure contents

I. QUESTION

```
1 int algorithm(int N){
2   if (N == 1)
3     return 0;
4   int sum = 0;
5   for (int i = N; i > 0; i--)
6     for (int j = 0; j < N; j++)
7       sum++;
8   for (int i = 0; i < 5; i++)
9     for (int j = 0; j < 5; j++)
10      sum += algorithm(N / 5);
11   return sum;
12 }
```

What is the time complexity of algorithm? You must construct the recurrence equation and solve it without using the master theorem.

II. QUESTION

Write a method

```
LinkedList calculateCounts()
```

which calculates the counts of each number in a sorted linked list. Linked list should contain a count for every number. You can use insertLast. Algorithm is as follows:

- Traverse the list
 - When the number is the first number, insert possibly leading zeros, initialize count.
 - When the number is the same as the previous number, increment the count
 - If the number is the new, initialize the count, insert the count of the previous number and possibly leading zero counts.
- For the last number, insert the count.

Original list:

2 → 2 → 5 → 5 → 5 → 7.

Output list:

0 → 2 → 0 → 0 → 3 → 0 → 1

III. QUESTION

Write a recursive method

```
bool averageOfItsChildren()
```

which returns true if there is at least one node which is the average of its children (left and right children must exist).

IV. QUESTION

Given the adjacency matrix representation of a graph G, write a method which returns true if there are two nodes whose outgoing node lists are the same, false otherwise. Your method should run in $\mathcal{O}(V^3)$ time.

```
bool outgoingListSame()
```

V. QUESTION

Modify the original insertion sort such that

```
void sort2(int* A, int* B, int N)
```

uses B as a secondary key. If two elements in A are equal, algorithm uses B as a secondary key. Modify array B also. N is the number of elements in the array. Your modified method should run in $\mathcal{O}(N^2)$ time.

VI. QUESTION

Given an array A of N integers, write a function

```
void topThreeFrequent(int* A, int N)
```

Your function should display the three most frequent elements in the given array A, which is already sorted. Your function must run in $\mathcal{O}(N)$ time. You should not modify the input array A.